

SECTION 3: INTRODUCTION TO QUERY OPTIMIZATION

Lab 1: Triggering nested loop join.

- Run these diagnostics:

Query Query History

```
1 --Step 1: Execute this and observe the "Nested Loop" node
2 EXPLAIN (ANALYZE, BUFFERS)
3 SELECT c.first_name, c.last_name, r.rental_date
4 FROM customer c
5 JOIN rental r ON c.customer_id = r.customer_id
6 WHERE c.customer_id = 1;
```

- Output:

Showing rows: 1 to 14				Page No: 1	of 1					
	QUERY PLAN									
	text									
1	Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=5.526..11.341 rows=32.00 loops=1)									
2	Buffers: shared read=153									
3	-> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.420..0.424 rows=1.00 loo...									
4	Index Cond: (customer_id = 1)									
5	Index Searches: 1									
6	Buffers: shared read=3									
7	-> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=5.094..10.887 rows=32.00 loops=1)									
8	Filter: (customer_id = 1)									
9	Rows Removed by Filter: 16012									
10	Buffers: shared read=150									
11	Planning:									
12	Buffers: shared hit=121 read=7 dirtied=1									
13	Planning Time: 13.621 ms									
14	Execution Time: 11.470 ms									

- Analysis question:

- **Identify the Roles:** Look at the plan tree. Which table is the Outer (top) and which is the Inner (bottom)?

Answer:

+ Outer Table (top): The `customer` table. In the execution plan (row 3), the Index Scan on `customer c` is the first branch directly under the Nested Loop node. The optimizer prioritizes the table with the smallest expected row count as the outer table.

+ Inner Table (bottom): The `rental` table. The Seq Scan on `rental r` (row 7) is the second branch. The Nested Loop algorithm takes the row from the outer table and scans this inner table to find matching records.

- **Why Nested Loop?** How many rows did the customer table return? Why does this make Hash Join “too expensive”?

Answer:

+ Rows returned by the `customer` table: The execution plan shows the index scan on the `customer` table returned exactly 1 row (row 3: `actual time=0.420..0.424 rows=1.00`).

+ Why Hash Join is "too expensive": A Hash Join requires a "Build" phase where the system allocates RAM (`work_mem`) to construct a hash table. Incurring the high startup cost of memory allocation and hashing for a single row is highly inefficient. A Nested Loop avoids this overhead entirely, making it much cheaper and faster for single-row lookups.

- **Efficiency:** Look at loops in the Inner Table node. Why is it 1?

Answer:

+ The `loops` metric indicates how many times the inner node (the `rental` table scan) was executed. Since the outer table (`customer`) only provided 1 row to the loop, the system only needed to execute the inner scan 1 time. During this single pass, it discarded 16,012 irrelevant records (row 9: `Rows Removed by Filter: 16012`) and successfully found the 32 matching rentals (row 7: `rows=32.00`).

Lab 2: Triggering Hash Join.

- Run these diagnostics:

Query	Query History
1	--Run this and look for "Hash Join" and "Hash" nodes
2	EXPLAIN (ANALYZE, BUFFERS)
3	SELECT f. title , l .name as language
4	FROM film f
5	JOIN language l ON f.language_id = l .language_id;

- Output:

	QUERY PLAN text	🔒
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=6.624..14.093 rows=1000.00 loops=1)	
2	Hash Cond: (f.language_id = l.language_id)	
3	Buffers: shared read=89	
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=4.079..11.066 rows=1000.00 lo...	
5	Buffers: shared read=88	
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=2.516..2.518 rows=6.00 loops=1)	
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
8	Buffers: shared read=1	
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=2.481..2.487 rows=6.00 loo...	
10	Buffers: shared read=1	
11	Planning:	
12	Buffers: shared hit=344 read=13	
13	Planning Time: 42.036 ms	
14	Execution Time: 17.248 ms	

- Analysis question:

- **Identify phases:** Which table is being Hashed (Built)? Which table is being Probed?

Answer:

+ Hashed (Build) table: The `language` table. Lines 6 and 9 show the Seq Scan on `language l` feeding directly into the Hash node to build the in-memory hash table.

+ Probed table: The `film` table. Line 4 shows the Seq Scan on `film f` acting as the outer table, which is scanned to probe against the hash table.

- **Memory check:** Look at the Hash node. What is the Memory Usage?

Answer:

+ On line 7, under the Hash node, the Memory Usage is 9kB. The `Batches: 1` metric indicates that the entire hash table fits completely into RAM.

- **Hypothesis:** What happens if you run `SET work_mem = '64kB';` and run it again? (Try it!)

Answer:

+ Run the hypothesis:

Query	Query History
1	<code>SET work_mem = '64kB';</code>
2	
3	<code>EXPLAIN (ANALYZE, BUFFERS)</code>
4	<code>SELECT f.title, l.name as language</code>
5	<code>FROM film f</code>
6	<code>JOIN language l ON f.language_id = l.language_id;</code>

+ Output:

	QUERY PLAN text
1	Merge Join (cost=1.29..127.82 rows=1000 width=99) (actual time=0.145..2.398 rows=1000.00 loops=1)
2	Merge Cond: (f.language_id = l.language_id)
3	Buffers: shared hit=56
4	-> Index Scan using idx_fk_language_id on film f (cost=0.15..114.15 rows=1000 width=17) (actual time=0.040..0.920 rows=1000.00 loo...
5	Index Searches: 1
6	Buffers: shared hit=55
7	-> Sort (cost=1.14..1.15 rows=6 width=88) (actual time=0.094..0.096 rows=1.00 loops=1)
8	Sort Key: l.language_id
9	Sort Method: quicksort Memory: 25kB
10	Buffers: shared hit=1
11	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.059..0.064 rows=6.00 loops=1)
12	Buffers: shared hit=1
13	Planning Time: 0.583 ms
14	Execution Time: 2.672 ms

+ The execution plan did change. By lowering `work_mem` to 64kB, the Optimizer switched from a Hash Join to a Merge Join. It determined that using an Index Scan and a 25kB in-memory `quicksort` was the optimal strategy under the new strict memory constraint.

Lab 3: Triggering Merge Join

- Run these diagnostics:

Query	Query History
1	<code>--Hint: Sometimes we must nudge the optimizer if tables are small</code>
2	<code>SET enable_hashjoin = OFF;</code>
3	<code>SET enable_nestloop = OFF;</code>
4	
5	<code>EXPLAIN (ANALYZE, BUFFERS)</code>
6	<code>SELECT f.film_id, f.title, i.inventory_id</code>
7	<code>FROM film f</code>
8	<code>JOIN inventory i ON f.film_id = i.film_id</code>
9	<code>ORDER BY f.film_id;</code>
10	<code>--Explicit sort requirement</code>

- Output:

	QUERY PLAN	
	text	🔒
1	Merge Join (cost=349.64..547.91 rows=4581 width=23) (actual time=4.004..7.243 rows=4581.00 loops=1)	
2	Merge Cond: (f.film_id = i.film_id)	
3	Buffers: shared hit=109	
4	-> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.059..0.679 rows=1000.00 loo...	
5	Index Searches: 1	
6	Buffers: shared hit=84	
7	-> Sort (cost=349.37..360.82 rows=4581 width=6) (actual time=3.928..4.417 rows=4581.00 loops=1)	
8	Sort Key: i.film_id	
9	Sort Method: quicksort Memory: 300kB	
10	Buffers: shared hit=25	
11	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.061..1.832 rows=4581.00 loops=1)	
12	Buffers: shared hit=25	
13	Planning:	
14	Buffers: shared hit=6	
15	Planning Time: 1.214 ms	
16	Execution Time: 8.049 ms	

- Analysis question:

- **Identify phases:** Do you see a Sort node? Or did it use Index Scan to get sorted data for free?

Answer:

+ You will see an Index Scan rather than an explicit `Sort` node. Because the join key (`film_id`) is indexed on these tables, the Optimizer knows the data in the index is already logically sorted. It uses the Index Scan to stream this pre-sorted data directly into the Merge Join, getting the ordering "for free" and bypassing the expensive sorting phase

- **Compare:** Reset the settings (`RESET ALL;`) and remove the `ORDER BY`. Does it switch back to Hash Join? Why?

Answer:

+ Yes, it will switch back to a Hash Join. When you execute `RESET ALL` and remove the `ORDER BY` requirement, you are returning control to the Cost-Based Optimizer. For joining two full tables without restrictive `WHERE` filters, the Optimizer calculates that allocating a small amount of memory to build a Hash Table (Hash Join) has a lower overall cost than sequentially traversing B-Tree indexes for every single row.